



A CLAUDE CODE PLUGIN • AUTONOMOUS ISSUE BATCHING

ticketmill

Batch-process GitHub
issues end to end,
autonomously. With the
paranoia built in.

Built by aaddrick

github.com/aaddrick/ticketmill
linkedin.com/in/aaddrick

SWIPE →

THE PREMISE

Autonomous coding is easy to demo. Trusting it is the hard part.

An agent that closes one issue in a clean video is not the problem worth solving. A batch of real issues, implemented while you sleep, with nothing unreviewed landing on your base branch, is.

MOST DEMOS

One issue. Happy path. Merge and hope.

TICKETMILL

A batch. Adversarial review at every gate. One PR a human merges.

THE OUTPUT

Every run ends with one PR, and a human owns the merge.

Each issue is researched, planned, built, tested, reviewed, and squash-merged into a throwaway `Batch_<ts>` branch. The run produces exactly one `Batch` → `base` PR. The engine never merges to your base branch itself.

VERIFICATION GAPS — ON EVERY BATCH PR

Tests disabled, browser skipped, an agent missing: every check that did not run is listed in front of the reviewer, not buried in the logs.

THE PIPELINE

Five stages. Every unit of work is a checked subagent call.

SELECT	Load the repo profile, claim issues so batches never collide, preflight-route each one: skip, review-only, or implement.
PLAN	Research, evaluate an approach, plan the tasks. A contrarian gate challenges both before a line of code exists.
BUILD	Implement each task with a review loop and a quality pass, then a test loop that halts on error, never skips.
SHIP	Open the per-issue PR, run spec and code review in parallel, then squash-merge and file follow-up issues.
REPORT	One batch PR to base, a machine- and human-readable run report, and a retrospective that feeds the next run.

DEVIL'S ADVOCATE

Before any code is written, someone argues against the plan.

Two gates stress-test the work: one challenges the approach, one the task plan. Each steel-mans the proposal first, then audits assumptions. The hard part is making dissent that does not thrash:

Earned severity

Critical means data loss or security; major means wrong behavior. Zero findings is an expected verdict for a trivial plan, not a failure to try.

A finding is a hypothesis

Every objection is verified against the actual tree before it is adopted, so a wrong "major" gets rebutted with evidence instead of blindly obeyed.

Settled ledger

Once a trade-off is adjudicated it travels forward under a "re-open only with new evidence" contract, so the gates stop re-litigating and burning model time.

THE HONEST GATE

Tests must pass, and the tests themselves are audited.

No profile, no run: the engine refuses to guess a toolchain, because a wrong guess would squash-merge untested work behind a green-looking PR.

LAYER 1 • THE SUITE RUNS

The loop halts loudly if the runner dies, a silent skip ships broken code, and it fixes the real defect, never weakens an assertion to turn red green.

LAYER 2 • THE TESTS ARE REAL

A separate validator audits for hollow assertions, mock abuse, and missing edge cases: tests that pass without ever exercising the change.

`test_command: null` is accepted only as a recorded human decision, and still shows as a gap on every batch PR.

CONTEXT THREADING

An agent four stages downstream still knows why.

Long pipelines forget. Four separate memories thread through every stage, so a late agent inherits the original intent, not just the last reviewer's complaint.

Decision chain

Distilled per-stage summaries, injected into every judgment gate.

Settled ledger

Adjudicated trade-offs, carried forward with their rationale.

Handoff notes

Env quirks and gotchas one agent found, handed to the next.

Learnings digest

Durable lessons from past runs, injected per stage.

RUNS IN PARALLEL, SAFELY

Many pipelines, one tracker, zero crossed wires.

Issues run concurrently, and two maintainers can start batches on two machines. The engine assumes collisions and prevents them mechanically, not by hoping.

- **Scope markers.** Every comment carries a machine-checkable marker; the gates delete any comment misfiled onto the wrong issue by a neighbor.
- **Cross-run claims.** Each issue is claimed before work starts; a fresh foreign claim is skipped, and simultaneous starts resolve by earliest timestamp.
- **Label safety.** A claim touches only its own label, never rewrites an issue's label set. A real incident, now a hard rule.

SURVIVES REALITY

Re-run the exact same command. It heals itself.

Power loss, usage limits, an outage. Recovery is not a special mode: the Select-phase preflight reads live GitHub and git state and re-routes every issue.

MERGED

already done, so skip it.

OPEN PR

go straight to review and merge.

PARTIAL

keep building from the commits already there.

Circuit breakers: three failed issues, or three consecutive agent deaths (the usage-limit signature), stop the run with a resume plan instead of burning the batch.

STACK-AGNOSTIC

It learns your toolchain and staffs the pipeline with your own agents.

A per-repo profile records how to install, test, and provision.

Roles map to the agents already in your `.claude/agents/`. One mechanism runs them all: each stage reads the agent file and adopts the persona, so a freshly forged agent works in the very next run, identically before and after a restart.

implementers
one per domain

contrarian
devil's-advocate gate

spec + code review
parallel PR gate

test_validator
audits tests

simplifier
refines, no behavior change

doc writers
docblocks + tech docs

Nothing fits a role? It runs on a built-in charter, or forges a project-specific agent grounded in your codebase.

MODEL POLICY

Opus where judgment lives. Haiku where it doesn't.

Every stage is assigned a model by how much judgment it actually needs. Spend is deliberate, and any stage is overridable in the profile.

haiku • low

Mechanical gh and git probes, worktree setup, running the test suite.

sonnet

Research, implementation, fixes, per-task and spec review, docs.

opus • high

The judgment gates: evaluate, plan, contrarian challenges, final code review.

IT GETS BETTER

Each run teaches the next one.

Every run distills durable learnings into a retrospective and injects them into the next run's planning, review, and test prompts, drawn from each run's structured evidence, not git archaeology.

NOTHING GETS FORGOTTEN

Work the run saw but deliberately did not do, out-of-scope ideas, deferred fixes, skipped reviews, becomes new, labeled follow-up issues at merge time.

EARNED, NOT DESIGNED

Every guardrail answers a specific incident.

MECHANISM

Comment markers + misfiled-comment deletion

Settled-decisions ledger

Test loop halts, never degrades

Claim label-safety rules

Circuit breakers + degrade windows

INCIDENT IT ANSWERS

A pipeline posted one issue's plan onto another issue.

Contrarian gates oscillated, burning model time re-litigating.

Silent test skips shipped broken code.

A claim agent replaced an issue's entire label set.

Could not tell one flaky stage from systemic failure.

TRY IT

Run it against your own backlog.

```
$ claude plugin marketplace add aaddrick/ticketmill
/plugin install ticketmill@ticketmill
/ticketmill:mill-init
– then –
/ticketmill:mill "mill issues 701 and 702 against dev"
```

github.com/aaddrick/ticketmill

linkedin.com/in/aaddrick

Built by aaddrick • MIT licensed

STAR IT →